Your program is broken and many of your tests fail. Just as individual programs have their own variables, you need a way to isolate individual components of your program so that a variable name in one part of the program doesn't accidentally overwrite or *clobber* the same variable name in another part of the program. You need *scope*.

3.5.1. Lexical function scope

All variables are lexically scoped to the body of a function and are defined only inside that function. In this section you'll see that implicit variable definitions in CoffeeScript can help prevent name collisions and global variables that can be a common affliction in JavaScript programs.

Functions are like programs with their own variables. If you first define a variable inside a function, then it can't be used outside the function:

```
scoped = ->
  secret = 'A secret'

secret
# ReferenceError: secret is not defined
```

secret is defined inside function

A variable that's scoped to a function is undefined outside of the function. This means you can have one part of the program where you use the name `accumulate` for one variable and another part of the program where Robin uses the name `accumulate` for a different variable:

```
subProgramOne = ->
  accumulate = (initial, numbers, accumulator) ->
    total = initial or 0
    for number in numbers
      total = accumulator total, number
    total

subProgramTwo = ->
  accumulate = yes
```

The two uses of `accumulate` in this example are different variables; functions provide scope.